머신러닝 프로그래밍
Machine Learning Programming

Lecture 2 : 데이터 이해, 전처리, 시각화

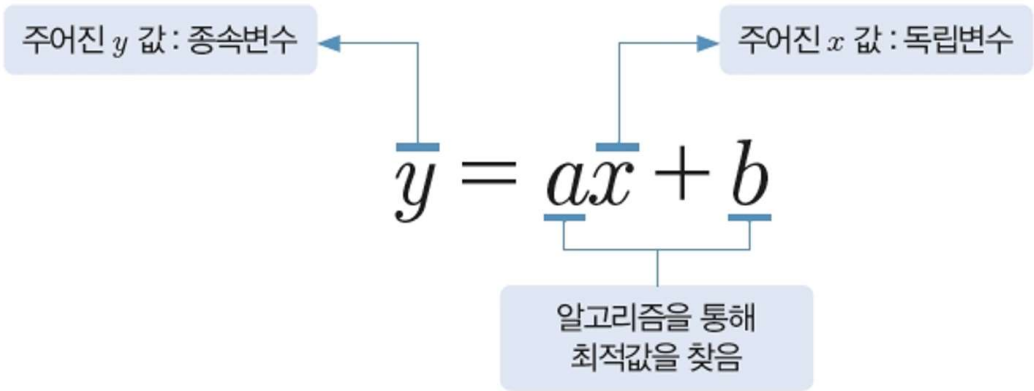
충북대학교
CHUNGBUK NATIONAL UNIVERSITY
충북대학교
산업인공지능연구센터

데이터의 이해 _ feature

1. feature
   - 개념: ‘특성’이나 ‘특징’이라는 의미
   - 예측의 근거가 되는 재료

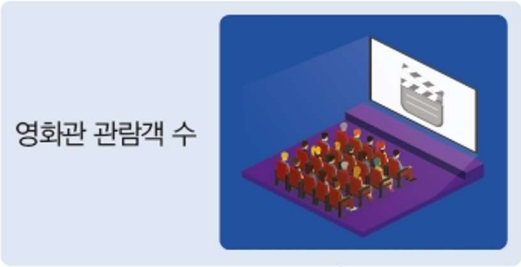
* 모델은 함수 형태로 표현

$$y = f(x)$$



데이터의 이해 _ feature

- 영화관 관람객 수를 인터넷 웹사이트의 좋아요 개수로 예측할 수 있다고 가정하면,



$$y = f(x)$$

데이터의 이해 _ feature

- 보스턴 집값 예측(Boston House Price) 데이터셋

x 변수 13개	[01] CRIM	자치시(town)별 1인당 범죄율
	[02] ZN	25,000 평방피트를 초과하는 거주지역의 비율
	[03] INDUS	비소매상업지역이 점유하고 있는 토지의 비율
	[04] CHAS	찰스강에 대한 더미변수(강의 경계에 위치한 경우는 1, 아니면 0)
	[05] NOX	10ppm 당 농축 일산화질소
	[06] RM	주택 1가구당 평균 방의 개수
	[07] AGE	1940년 이전에 건축된 소유주택의 비율
	[08] DIS	5개의 보스턴 직업센터까지의 접근성 지수
	[09] RAD	방사형 도로까지의 접근성 지수
	[10] TAX	10,000달러 당 재산세율
	[11] PTRATIO	자치시(town)별 학생/교사 비율
	[12] B	1000(Bk−0.63)^2, 여기서 Bk는 자치시별 흑인의 비율을 말함
	[13] LSTAT	모집단의 하위계층의 비율(%)
y 변수	[14] MEDV	본인 소유의 주택가격(중앙값) (단위 : $1,000)

데이터의 이해 _ feature

- 범죄율, 방의 개수, 재산세율 등의 값들이 어떻게 집값(종속변수)에 영향을 주는지에 대한 모델
- 서로 다른 독립변수 13개를 x_n로 하고 가중치 β_n을 선형 결합(linear combination)하여 나타냄

$$y = \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \beta_4 x_4 + \cdots + \beta_{13} x_{13} + \beta_0 \times 1$$

[TIP] 위 표현식 중 β_n은 통계학에서 흔히 사용하는 표현이다. 일반적으로 머신러닝에서는 가중치를 의미하는 w을 많이 사용한다.

데이터의 이해 _ feature

- 데이터 테이블(data table) : 데이터를 테이블(표)로 표현한 것
  - 모판 : 현업에서 '데이터의 판'이라는 의미로 사용
- Feature : 데이터 테이블 상에서 하나의 열 이름(column name)

CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV	CAT.MEDV
0.00632	18	2.31	0	0.538	6.575	65.2	4.09	1	296	15.3	396.9	4.98	24	0
0.02731	0	7.07	0	0.469	6.421	48.9	4.9671	2	242	17.8	396.9	9.14	21.6	0
0.02729	0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7	1
0.03237	0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4	1
0.06905	0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.9	5.33	36.2	1
0.02985	0	2.18	0	0.458	6.43	58.7	6.0622	3	222	18.7	394.12	5.21	28.7	0
0.08829	12.5	2.18	0	0.524	6.012	66.6	5.5605	5	311	15.2	395.6	12.43	22.9	0
0.14455	12.5	7.87	0	0.524	6.172	96.1	5.9505	5	311	15.2	396.9	19.15	27.1	0
0.21124	12.5	7.87	0	0.524	5.631	100	6.0821	5	311	15.2	386.63	29.93	16.5	0
0.17004	12.5	7.87	0	0.524	6.004	85.9	6.5921	5	311	15.2	386.71	17.1	18.9	0
0.22489	12.5	7.87	0	0.524	6.377	94.3	6.3467	5	311	15.2	392.52	20.45	15	0
0.11747	12.5	7.87	0	0.524	6.009	82.9	6.2267	5	311	15.2	396.9	13.27	18.9	0
0.09378	12.5	7.87	0	0.524	5.889	39	5.4509	5	311	15.2	390.5	15.71	21.7	0
0.65976	0	8.14	0	0.538	5.949	61.8	4.7075	4	307	21	396.9	8.26	20.4	0
0.63796	0	8.14	0	0.538	6.096	84.5	6.4619	4	307	21	380.02	10.26	18.2	0

데이터의 이해 _ feature

- 데이터 인스턴스(data instance) : 하나의 데이터
  - 튜플(tuple)이라고도 부름
  - 엑셀에서의 한 줄 한 줄

CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV	CAT.MEDV
0.00632	18	2.31	0	0.538	6.575	65.2	4.09	1	296	15.3	396.9	4.98	24	0

$$y = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + \cdots + w_{13}x_{13} + w_0 \times 1$$

데이터의 이해 _ feature

벡터 형태로 표현

$$x^{(1)} = \begin{bmatrix} 1 \\ 0.00632 \\ 18 \\ 2.31 \\ 0.538 \\ \vdots \\ 4.98 \end{bmatrix} \qquad w = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ w_3 \\ \vdots \\ w_{13} \end{bmatrix}$$

- 하나의 행(row), 즉 튜플의 값은 열 벡터(column vector)로 표현하고, 가중치(weight)도 열 벡터로 표현

- ${x_j}^{(i)}$에서 i 는 전체 데이터 테이블에서 데이터의 순서, j는 피쳐의 순서

x_6^2 = ?

데이터의 이해 _ feature

데이터의 분류

1. Continuous data : 값이 끊어지지 않고 계속 연결되는 종류의 데이터로 실수와 관련된 값
   - 온도, 자동차의 속도
   - 평균, 분산 등 통계적 기법 적용 가능

2. Discrete data : 분리해서 표현하는 데이터, 일종의 라벨로 구분 가능
   - 우편번호(숫자로 표현), 성별(텍스트로 표현)
   - 텍스트 형태의 값도 숫자 형태로 바꾸어 수식의 x_j에 할당

연속형 데이터와 이산형 데이터의 가장 큰 차이는 그 숫자의 의미가 스케일(scale)이 있는가 없는가 임.

데이터의 이해 _ feature

Why is the difference between continuous and discrete data important?

- 머신러닝 전처리 방법을 결정.

데이터 전처리

- 데이터 전처리 : raw data를 정리하고 변환하여 분석에 적합한 형태로 만드는 과정
  - 정확도 향상
  - 모델 성능 개선
  - 분석 효율성 증대

데이터 전처리

- 결측치 (missing data) 처리

  - 행삭제 (dropping rows)

  - 열삭제 (dropping columns)

  - 대체(imputation)

- 이상치(outlier) 처리

  데이터 내에서 다른 값들과 현저히 차이나는 극단적인 값

데이터 전처리

- 데이터 불러오기

```python
import pandas as pd
from pathlib import Path

def load_housing_data():
    # [방법 1] 상대 경로 방식 (이미지 가이드에 따른 추천 방식)
    # 현재 실행 중인 파이썬 파일 위치를 기준으로 datasets 폴더 탐색
    base_path = Path(__file__).parent
    csv_path = base_path / "datasets" / "housing.csv"

    # [방법 2] 절대 경로 방식 (직접 경로를 지정할 학생만 주석 해제 후 사용)
    # csv_path = Path(r"C:\Users\LG\Desktop\lecture\week 2\project\datasets\housing.csv")

    # 데이터 로드
    return pd.read_csv(csv_path)

if __name__ == "__main__":
    # 1. 데이터 불러오기
    housing = load_housing_data()

    # 2. 데이터 확인 (상위 5개 행)
    print("--- Housing Data Head ---")
    print(housing.head())

    # 3. 데이터 요약 정보 확인
    print("\n--- Housing Data Info ---")
    print(housing.info())
```

데이터 전처리

- 결측치 (missing data) 처리

```python
import pandas as pd
from pathlib import Path

def load_housing_data():
    base_path = Path(__file__).parent
    csv_path = base_path / "datasets" / "housing.csv"
    return pd.read_csv(csv_path)

if __name__ == "__main__":
    # Load data
    housing = load_housing_data()

    # Step 1: Initial check
    print("\nStep 1: Checking for missing values")
    print(housing.isnull().sum())

    # Step 2: Processing
    print("\nStep 2: Selecting Missing Data Strategy")

    # [Option 1] Drop rows with missing values
    # housing = housing.dropna(subset=["total_bedrooms"])
    # print("Result: Dropped rows containing missing values.")

    # [Option 2] Drop the entire column
    # housing = housing.drop("total_bedrooms", axis=1)
    # print("Result: Dropped the 'total_bedrooms' column.")

    # [Option 3] Impute with median (Recommended)
    median = housing["total_bedrooms"].median()
    housing["total_bedrooms"] = housing["total_bedrooms"].fillna(median)
    print(f"Result: Imputed missing values with median ({median}).")

    # Step 3: Final verification
    print("\nStep 3: Verification after processing")
    print(housing.isnull().sum())
```

데이터 전처리

- 데이터 변환 (scaling & normalization)

  변수 간의 크기 차이를 조정하는 과정

  - 정규화 (normalization)
  - 표준화(standardization)

- 범주형 데이터 처리 (categorical encoding)

  특정 클래스나 그룹으로 구분되는 데이터이므로,

  수치형 데이터로 변환

  - 문자형 데이터를 숫자형으로 표현
  - 측정된 빈도 또는 통계적 특성에 기반하여 변환

데이터 전처리

데이터 변환

```python
import pandas as pd
from pathlib import Path
from sklearn.preprocessing import StandardScaler, MinMaxScaler


def load_housing_data():
    base_path = Path(__file__).parent
    csv_path = base_path / "datasets" / "housing.csv"
    return pd.read_csv(csv_path)


if __name__ == "__main__":
    # 1. Load Data
    housing = load_housing_data()

    # 2. Preprocessing: Handle Missing Values (Required before scaling)
    median = housing["total_bedrooms"].median()
    housing["total_bedrooms"] = housing["total_bedrooms"].fillna(median)

    # 3. Step 4: Check original statistics before scaling
    print("\nStep 4: Check original statistics before scaling")
    print("-" * 50)
    # Select only numeric features (Drop 'ocean_proximity' for scaling)
    num_housing = housing.drop( labels: "ocean_proximity", axis=1)
    print(num_housing.describe())

    # 4. Step 5: Scaling & Normalization
    std_scaler = StandardScaler()  # Standardization
    min_max_scaler = MinMaxScaler()  # Normalization

    # fit_transform returns a Numpy array
    housing_std = std_scaler.fit_transform(num_housing)
    housing_minmax = min_max_scaler.fit_transform(num_housing)

    print("\nStep 5: Compare results of Scaling (median_income column)")
    print("-" * 50)
    # median_income is at index 7
    print(f"Original value:      {num_housing['median_income'].iloc[0]}")
    print(f"Standardized result: {housing_std[0, 7]:.4f}")
    print(f"Normalized result:   {housing_minmax[0, 7]:.4f}")

    # 5. Step 6: Min/Max verification
    print("\nStep 6: Min/Max values after Normalization")
    print("-" * 50)
    print(f"Min value (All features): {housing_minmax.min()}")
    print(f"Max value (All features): {housing_minmax.max()}")
```

데이터 전처리

- 중복 데이터 처리 (duplicates handling)

  행이 반복적으로 나타나는 경우 등

번호	품목	원산지	재질	개수	
1	스프링	한국	철	150	
2	망치	대만	철, 나무	200	
3	줄자	영국	철	100	
4	수평계	독일	알루미늄	120	
5	스프링	한국	철	150	◀---- 완전 중복
6	대패	미국	나무, 철	100	
7	스프링	한국	나무	150	◀---- 부분 중복
8	렌치	캐나다	철	110	

데이터 전처리

- 데이터 분할(train-test split)

  학습용 데이터 : 검증용 데이터 : 테스트 데이터

- 하이퍼파라미터 튜닝(hyperparameter tuning)

  모델 성능을 최적화하기 위해 모델을 학습하는 과정에서

  외부에서 설정하는 값

데이터 전처리

- 데이터 분할(train-test split)

```python
import pandas as pd
from pathlib import Path
from sklearn.model_selection import train_test_split

def load_housing_data():
    base_path = Path(__file__).parent
    csv_path = base_path / "datasets" / "housing.csv"
    return pd.read_csv(csv_path)

if __name__ == "__main__":
    # 1. 데이터 로드
    housing = load_housing_data()
    base_path = Path(__file__).parent
    output_dir = base_path / "datasets" # 저장할 폴더 위치

    # 2. 데이터 분할 (6:2:2)
    # [Step A] 전체의 20%를 Test 세트로 분리
    train_valid_set, test_set = train_test_split(
        *arrays: housing, test_size=0.2, random_state=42
    )

    # [Step B] 남은 80% 중 25%를 Validation 세트로 분리 (전체의 20% 효과)
    train_set, valid_set = train_test_split(
        *arrays: train_valid_set, test_size=0.25, random_state=42
    )

    # 3. 분할된 결과 출력 (검증)
    print("\nStep 9: Raw Data Splitting Results (6:2:2)")
    print("-" * 50)
    print(f"Total dataset:      {len(housing)} rows")
    print(f"Training set (60%): {len(train_set)} rows")
    print(f"Validation set (20%): {len(valid_set)} rows")
    print(f"Test set (20%):       {len(test_set)} rows")

    # 4. 파일로 저장 (index=False 옵션으로 불필요한 번호 생성을 막습니다)
    train_set.to_csv(output_dir / "housing_train.csv", index=False)
    valid_set.to_csv(output_dir / "housing_valid.csv", index=False)
    test_set.to_csv(output_dir / "housing_test.csv", index=False)

    print("\nStep 10: Saving Files Complete")
    print("-" * 50)
    print(f"Files saved in: {output_dir}")
```

데이터 전처리

파라미터 vs 하이퍼파라미터

- 파라미터 : 모델 내부에서 결정되는 값으로,

훈련 데이터 (Training Data)를 통해 자동으로 최적화

- ex. 선형 회귀: 가중치, 편향

- 하이퍼파라미터 : 모델 학습을 시작하기 전에 사용자가 직접 설정하는 외부 구성 값으로 실험을 통해 최적의 값을 찾아야 함

- ex. 학습률, 배치크기, 반복횟수

데이터 전처리

실습 코드 관리

My_project /　　(프로젝트 폴더)

- OOO_1. py
- OOO_2. py
- OOO_3. py

- datasets/ (폴더 생성)
  - housing.csv (데이터 위치)